NBA Statline Forecasting

Context-Aware Machine Learning for Player Performance Prediction

Arul Selvakumar
DSCI 4350: Data Science Capstone

Predicting NBA player statlines using pregame contextual features and deployable ML systems.

Problem Statement

- Statline forecasting is difficult due to dynamic game context
- Traditional use of season averages as prediction factors ignores opponent, usage, and availability factors
- Need accurate pregame predictions with no data leakage
- Applications: analytics, betting markets, decision systems
- **Challenge**: capturing pregame context without introducing data leakage

Why This Matters

Performance varies game-to-game. Context-aware models can capture patterns that static averages miss.

Introduction

Goal

- Predict Points, Rebounds, Assists using pregame features

Approach

- Context-aware feature engineering
- Compare linear vs nonlinear models
- Optimize via hyperparameter tuning (V4)

Key Result

Significant MAE reduction achieved after V3 feature engineering + V4 hyperparameter tuning.

Pipeline: data → modeling → evaluation → deployment

Data Collection — Overview

~52,707	694	~2,460
Player-Game Rows	**Unique Players**	**Total Games**
across 2 seasons	in dataset	regular season

Source: nba_api (Python)

Train: 2023–24 season **Test:** 2024–25 season

- Time-based split (no random sampling) to prevent data leakage
- Player-level granularity (one row per player per game)

Data Collection — Features

Feature Groups

- Season averages
- Rolling metrics (L5 / L10)
- Opponent context (pace, defense)
- Availability (missing production)
- Feature scaling not required (tree-based models)

Target Variables

PTS — Points

REB — Rebounds

AST — Assists

Only pregame features used — no in-game data leakage

Modeling

Linear Regression

Baseline

Interpretable reference point

Random Forest

Ensemble

Captures nonlinear relationships

XGBoost

Gradient Boosting

Best predictive performance

Goal: Compare interpretability vs predictive performance across model complexity

Linear regression used as interpretability baseline. XGBoost chosen for final deployment due to performance.

Feature Versioning

Baseline	Season averages only
V0	+ Opponent context + Home/Away
V1	+ Availability / missing production
V2	+ Usage rate + matchup context
V3	+ Interaction features (usage × minutes, pace-adjusted stats, trends) Introduces nonlinear interaction features capturing further game context.
V4	Hyperparameter tuning on V3 features

Evaluation Methodology

Time-Based Holdout

- Train: 2023–24 season
- Test: 2024–25 season (unseen)

Evaluation strictly performed on future season (2024-25).

Ensures real-world generalization.

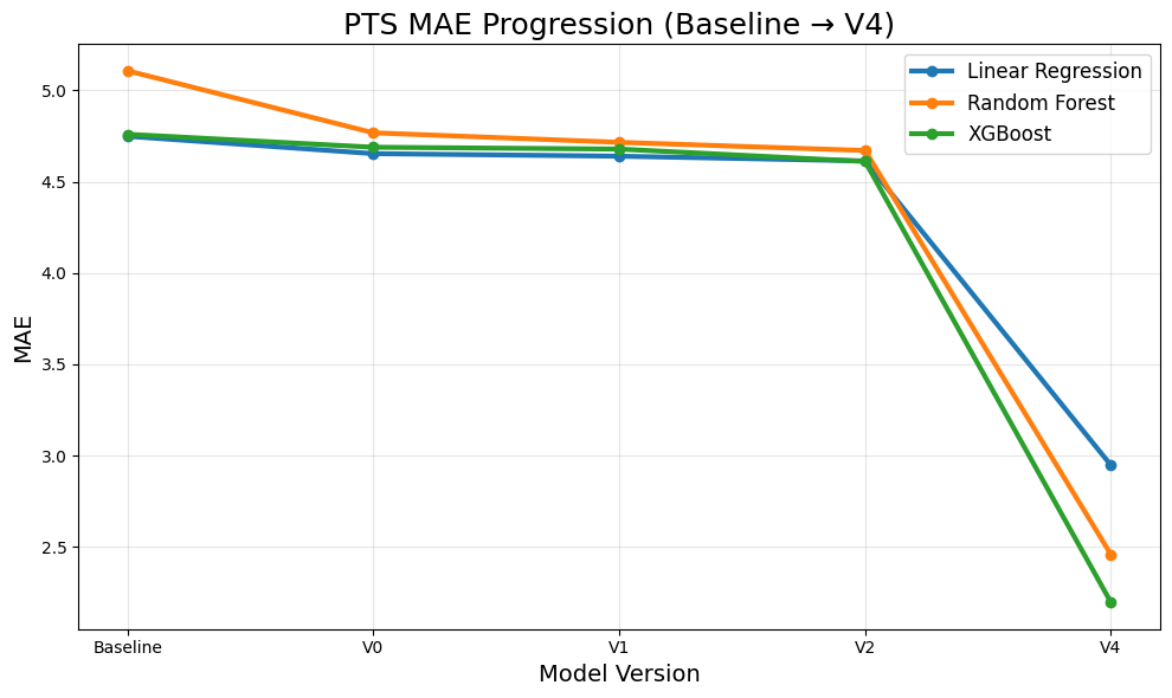
Evaluation Metrics

MAE — Primary metric (mean absolute error)

RMSE — Secondary metric (root mean squared error)

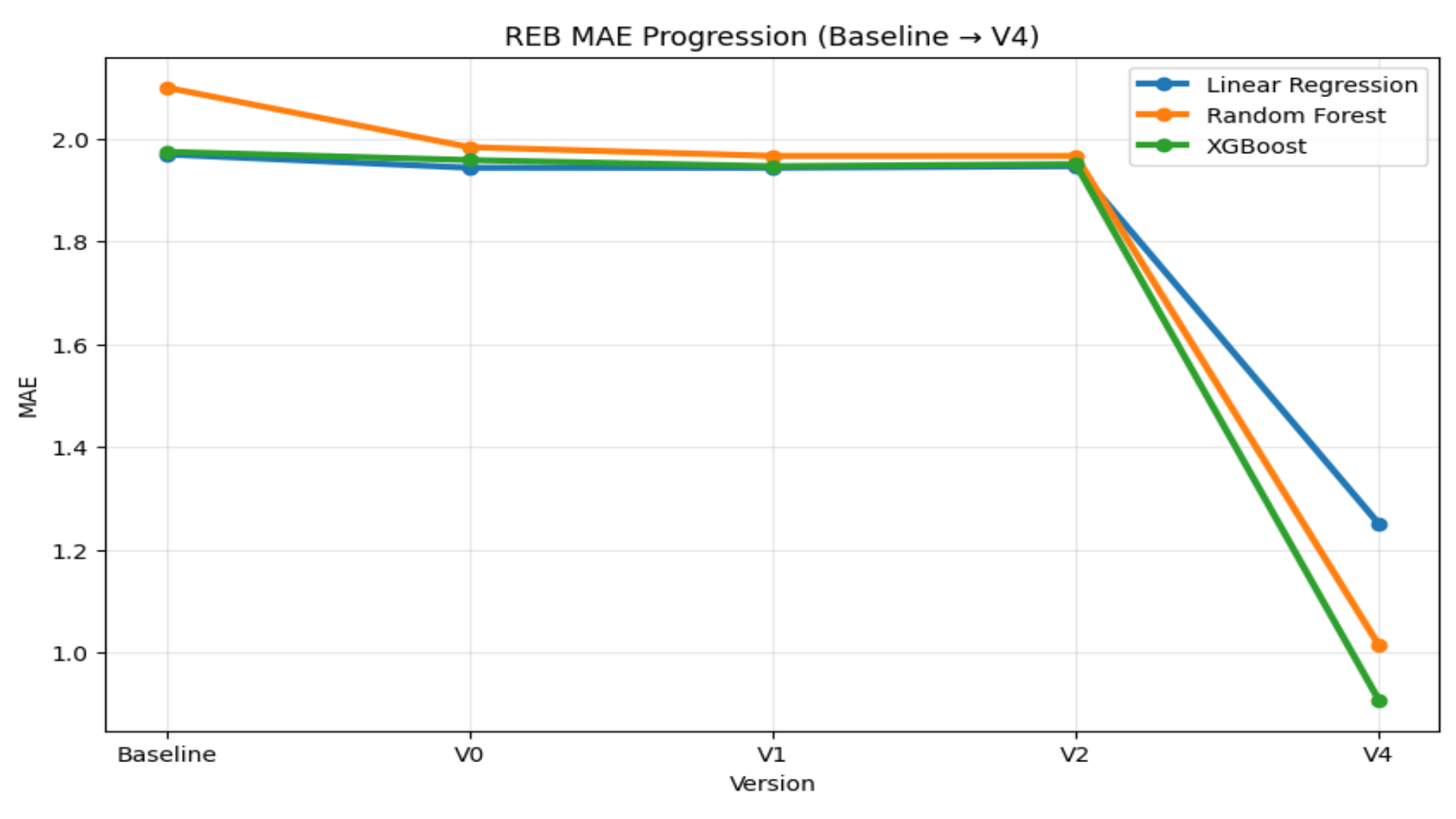
Goal: Minimize prediction error on unseen future season data

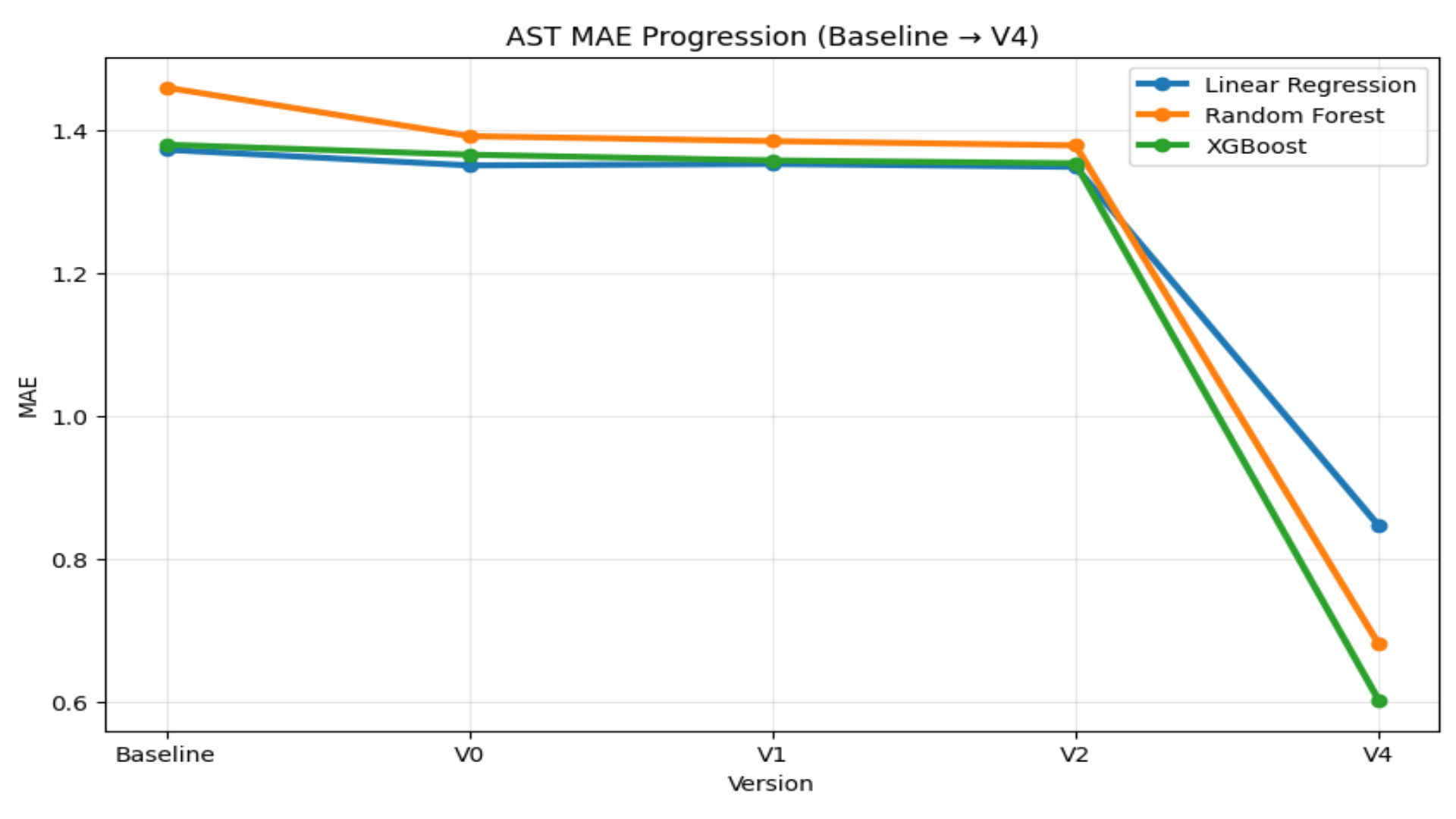
Results — MAE Progression



Key Observations

- Improvement from Baseline → V2
- Large performance jump at V4 after hyperparameter tuning
- Feature engineering + tuning significantly improve performance
- Major improvement with V3 features and V4 tuning.
- **V4 combines V3 engineering with hyperparameter tuning.**


REB MAE Progression (Baseline → V4)
Linear Regression
Random Forest
XGBoost
2.0
1.8
1.6
1.4
1.2
1.0
MAE
Baseline
V0
V1
V2
V4
Version


AST MAE Progression (Baseline → V4)
Linear Regression
Random Forest
XGBoost
MAE
1.4
1.2
1.0
0.8
0.6
Baseline
V0
V1
V2
V4
Version

```
======================================================================
Target: target_pts
======================================================================

Training Linear Regression reference...
C:\Users\aruls\AppData\Local\Temp\ipykernel_29884\1853218148.py:59: Pandas4Warning: For backward compatibility,
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for detail
  categorical_cols = train_df[feature_cols].select_dtypes(include=["object","category"]).columns.tolist()
V4_target_pts_LinearRegression: MAE=2.9467, RMSE=3.9471

Tuning Random Forest...
Fitting 1 folds for each of 12 candidates, totalling 12 fits
Best RF params: {'model__n_estimators': 100, 'model__min_samples_split': 5, 'model__min_samples_leaf': 1, 'model
V4_target_pts_RandomForestRegressor: MAE=2.4576, RMSE=3.3878

Tuning XGBoost...
Fitting 1 folds for each of 15 candidates, totalling 15 fits
Best XGB params: {'model__subsample': 1.0, 'model__reg_lambda': 1, 'model__reg_alpha': 0, 'model__n_estimators':
V4_target_pts_XGBRegressor: MAE=2.1953, RMSE=3.0124
```

Final Results

Best Model: **XGBoost (V4)**

PTS MAE	REB MAE	AST MAE
≈ 2.19	≈ 0.91	≈ 0.60

Key Insights

- Nonlinear models outperform after feature improvements
- Performance gains consistent across all target variables
- No data leakage — evaluated on future season
- Feature engineering + tuning drive performance gains
- **XGBoost outperformed linear and ensemble baselines.**

Best models by target:

	run_id	version	target	model	tuned	mae	rmse	feature_count	runtime_seco
0	V4_target_ast_XGBRegressor	V4	target_ast	XGBRegressor	True	0.602005	0.886846	78	3.015
1	V4_target_reb_XGBRegressor	V4	target_reb	XGBRegressor	True	0.908092	1.257660	78	3.174
2	V4_target_pts_XGBRegressor	V4	target_pts	XGBRegressor	True	2.195287	3.012371	78	2.848

	run_id	version	target	model	tuned	mae	rmse
8	V4_target_ast_XGBRegressor	V4	target_ast	XGBRegressor	True	0.602005	0.886846
7	V4_target_ast_RandomForestRegressor	V4	target_ast	RandomForestRegressor	True	0.680376	1.002479
6	V4_target_ast_LinearRegression	V4	target_ast	LinearRegression	False	0.845995	1.193216
2	V4_target_pts_XGBRegressor	V4	target_pts	XGBRegressor	True	2.195287	3.012371
1	V4_target_pts_RandomForestRegressor	V4	target_pts	RandomForestRegressor	True	2.457605	3.387816
0	V4_target_pts_LinearRegression	V4	target_pts	LinearRegression	False	2.946712	3.947072
5	V4_target_reb_XGBRegressor	V4	target_reb	XGBRegressor	True	0.908092	1.257660
4	V4_target_reb_RandomForestRegressor	V4	target_reb	RandomForestRegressor	True	1.014672	1.404213
3	V4_target_reb_LinearRegression	V4	target_reb	LinearRegression	False	1.250617	1.691776

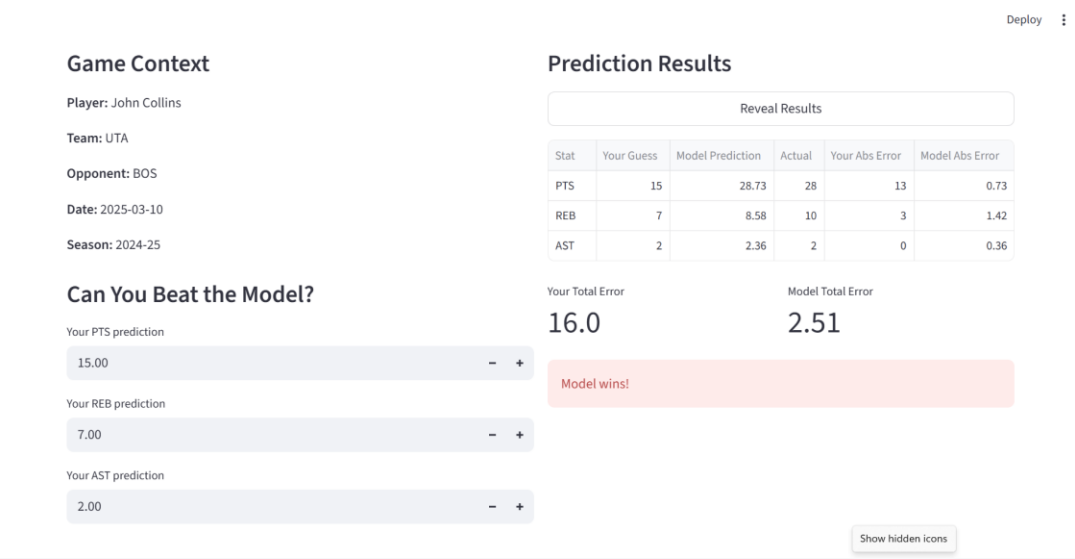
Deployment

- Streamlit web application
- Uses trained V4 XGBoost models
- Inputs: pregame features (opponent, usage, availability)
- Outputs: predicted statlines (PTS, REB, AST)
- Simulates real-world pregame inference pipeline for forecasting scenarios.

Interactive Demo

"Can You Beat the Model?"

- User inputs their own statline guesses
- System compares user vs model vs actual
- Outputs total error and declares a winner
- Engages audience with the prediction task



Monitoring & Governance

Monitoring

- Track MAE / RMSE over time
- Monitor feature drift (usage, lineup changes)
- Model versioning / experiment tracking
- Detect model drift due to roster and lineup changes
- Retraining triggered periodically (rolling updates)

Future Work

Neural networks (deep learning models)

Real-time API integration

Dashboard expansion